

系统开发工具基础 __ 实验报告（第 4 周）

李雨潼

25020007069

一、课上实验题

第 13 题：建立可执行的本地质量门禁

题目要求：复用第 10 题的 greetlab 包，在 q13 中建立轻量本地质量检查。在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。补充一个正常姓名测试和一个空白姓名测试。运行 ruff format、ruff check 和 pytest，修复全部问题。编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest，运行并保证返回 0。

执行结果：

check.sh 运行成功（返回 0）：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q13$ ./check.sh
echo "Exit code: $?"
2 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/dorothea/hw1/q13
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.4.4
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.09s =====
Exit code: 0
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q13$
```

图 1: check.sh 执行成功，返回 0

pytest 测试全部通过：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q13$ ruff format src/
ruff check src/
pytest -v
1 file reformatted, 1 file left unchanged
All checks passed!
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/dorothea/hw1/q13
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.4.4
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name PASSED [100%]

===== 2 passed in 0.14s =====
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q13$
```

图 2: pytest 测试通过（2 passed）

第 14 题：让 Make 只重建真正受影响的产物

题目要求：创建 data.csv、stats.py、build_report.py 和 report.md。编写 Makefile，包含 all、stats.txt、report.txt 和 clean。验证首次 make 生成两个产物；第二次无改动时不执行配方；touch data.csv 后再次 make，确认重新生成；clean 只删除生成物。

执行结果：

首次 make 生成两个产物：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$ make
ls -la stats.txt report.txt
python3 stats.py
python3 build_report.py
-rw-rw-r-- 1 dorothea dorothea 25 Sep  7 16:46 report.txt
-rw-rw-r-- 1 dorothea dorothea  2 Sep  7 16:46 stats.txt
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$
```

图 3: 首次 make 生成 stats.txt 和 report.txt

第二次 make 无改动时不执行配方：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$ make
make: 对“all”无需做任何事。
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$
```

图 4: 第二次 make 显示无需做任何事

touch data.csv 后 make 重新生成：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$ touch data.csv
make
python3 stats.py
python3 build_report.py
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$
```

图 5: 修改依赖后 make 重新生成

make clean 只删除生成物：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$ make clean
ls -la stats.txt report.txt
rm -f stats.txt report.txt
ls: cannot access 'stats.txt': No such file or directory
ls: cannot access 'report.txt': No such file or directory
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q14$
```

图 6: make clean 删除 stats.txt 和 report.txt

第 15 题：把本地 API 数据转换为可读报告

题目要求：在 q15 中创建 packages.json，运行本地 HTTP 服务。用 curl 获取 JSON，用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，按 downloads 降序、name 升序排列。编写 api_report.sh，生成 summary.md，包含标题和 name/version/downloads 三列表格。

执行结果：

curl + jq 筛选排序结果：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q15$ curl -fsS http://127.0.0.1:8000/packages.json | jq '.[] | select(.status == "active" and .downloads >= 100) | {name, version, downloads}' | jq -s 'sort_by(-.downloads, .name)'
127.0.0.1 - - [07/Sep/2026 16:57:28] "GET /packages.json HTTP/1.1" 200 -
[
  {
    "name": "delta",
    "version": "0.9.0",
    "downloads": 450
  },
  {
    "name": "gamma",
    "version": "1.5.1",
    "downloads": 450
  },
  {
    "name": "alpha",
    "version": "1.2.0",
    "downloads": 120
  }
]
```

图 7: curl + jq 筛选排序结果

summary.md 表格内容：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q15$ ./api_report.sh
cat summary.md
127.0.0.1 - - [07/Sep/2026 16:59:28] "GET /packages.json HTTP/1.1" 200 -
# Package Report

| name | version | downloads |
| --- | --- | ---: |
| gamma | 1.5.1 | 450 |
| delta | 0.9.0 | 450 |
| alpha | 1.2.0 | 120 |
显示应用
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q15$
```

图 8: summary.md Markdown 表格

第 16 题：修复并交付一个陌生的小型工具仓库

题目要求：复制 q13 为 q16，把 cli.py 改成写日志，运行 check.sh 确认测试失败。编写最小 Makefile，包含 check、build 和 clean。运行 make check 和 make build，计算 wheel 的 SHA-256。

执行结果：

故意破坏代码后 check.sh 失败：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$ ./check.sh
2 files already formatted
F401 [*] `sys` imported but unused
--> src/greetlab/cli.py:2:8
|
1 | import argparse
2 | import sys
|      ^^^
help: Remove unused import: `sys`
|
1 | import argparse
- import sys
2 |
|

Found 1 error.
[*] 1 fixable with the `--fix` option.
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$
```

图 9: check.sh 失败（F401 未使用导入）

修复后 check.sh 通过：

```
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$ ruff format src/
1 file reformatted, 1 file left unchanged
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$ ./check.sh
2 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/dorothea/hw1/q16
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.4.4
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.15s =====
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$
```

图 10: 修复后 check.sh 通过

make check 成功 + wheel SHA-256:

```

(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$ make check
make build
sha256sum dist/*.whl
./check.sh
2 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/dorothea/hw1/q16
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.4.4
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.11s =====
python3 -m build
/home/dorothea/hw1/hw3_practice/venv_ex1/bin/python3: No module named build
make: *** [Makefile:7: build] 错误 1
2831e5a4b877f17a4bf9a10592cbc5e50e7f18d6cf85df770948d8ccb51b3aa9 dist/greetlab_25020007069-0.1.0-py3-none-any.whl
(venv_ex1) dorothea@dorothea-VirtualBox:~/hw1/q16$

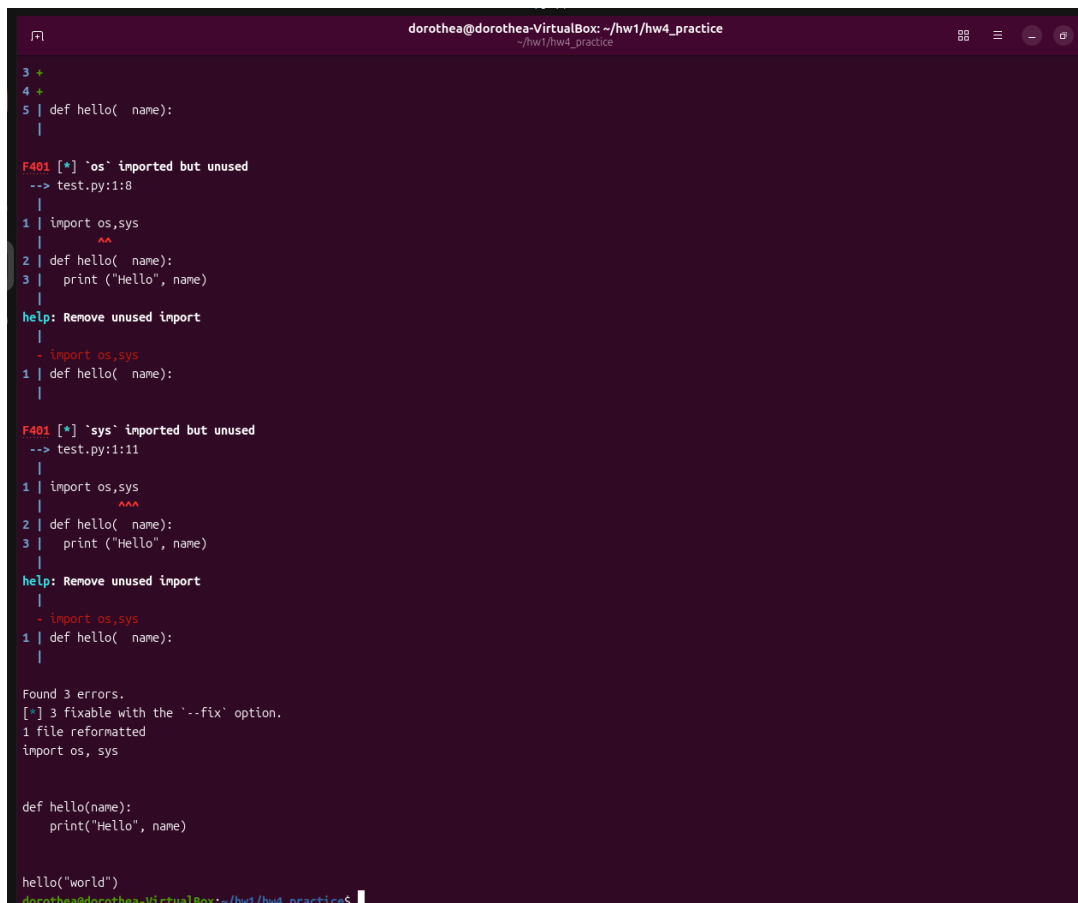
```

图 11: make check 成功及 wheel SHA-256

二、课后练习（10 个实例）

实例 1：配置 ruff 格式化器和 linter

运行 ruff check 和 ruff format，观察格式化前后的变化。



```
dorothea@dorothea-VirtualBox: ~/hw1/hw4_practice
3 +
4 +
5 | def hello( name):
6 |
7 |
F401 [*] `os` imported but unused
--> test.py:1:8
1 |
2 |
3 |
help: Remove unused import
- import os,sys
1 | def hello( name):
2 |
3 |
F401 [*] `sys` imported but unused
--> test.py:1:11
1 |
2 |
3 |
help: Remove unused import
- import os,sys
1 | def hello( name):
2 |
3 |
Found 3 errors.
[*] 3 fixable with the `--fix` option.
1 file reformatted
import os, sys

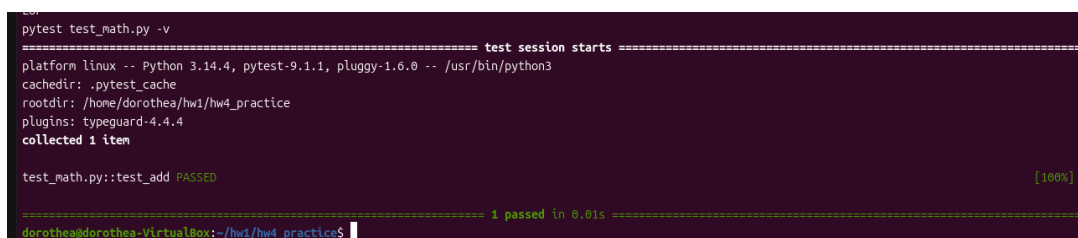
def hello(name):
    print("hello", name)

hello("world")
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 12: 实例 1 执行结果

实例 2：编写单元测试

用 pytest 编写并运行一个简单的加法函数测试。



```
pytest test_math.py -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/dorothea/hw1/hw4_practice
plugins: typeguard-4.4.4
collected 1 item

test_math.py::test_add PASSED [100%]

===== 1 passed in 0.01s =====
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 13: 实例 2 执行结果

实例 3：设置 pre-commit 钩子

创建.git/hooks/pre-commit 钩子，在提交前运行 ruff check。

```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ mkdir -p .git/hooks
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash
echo "Running pre-commit checks..."
ruff check .
EOF
chmod +x .git/hooks/pre-commit
ls -la .git/hooks/pre-commit
-rwxrwxr-x 1 dorothea dorothea 61 Sep  7 19:50 .git/hooks/pre-commit
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 14: 实例 3 执行结果

实例 4: 正则表达式查找 shell=True

用 grep 在代码中查找 subprocess.Popen(..., shell=True) 的使用。

```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ echo 'subprocess.Popen("ls", shell=True)' > test_code.py
grep -n "shell=True" test_code.py
1:subprocess.Popen("ls", shell=True)
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 15: 实例 4 执行结果

实例 5: Markdown 项目符号替换

用 sed 将 Markdown 文件中的 - 项目符号替换为 *。

```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ echo "- item1" > test.md
echo "- item2" >> test.md
echo "normal text" >> test.md
cat test.md
sed -i 's/^- /* /' test.md
cat test.md
- item1
- item2
normal text
* item1
* item2
normal text
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 16: 实例 5 执行结果

实例 6: JSON 提取 name 字段

用 Python json 模块从 JSON 结构中提取 name 字段。

```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ echo '{"name": "Alyssa P. Hacker", "college": "MIT"}' > data.json
python3 -c "import json, sys; data=json.load(sys.stdin); print(data['name'])" < data.json
Alyssa P. Hacker
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 17: 实例 6 执行结果

实例 7: Makefile 中的 clean 目标

编写 Makefile, 包含 all 和 clean 目标, 验证 clean 删除生成物。

```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ cat > Makefile << 'EOF'
.PHONY: all clean
all:
    touch output.txt
clean:
    rm -f output.txt
EOF
make all
ls -la output.txt
make clean
ls -la output.txt
touch output.txt
-rw-rw-r-- 1 dorothea dorothea 0 Sep  7 19:52 output.txt
rm -f output.txt
ls: cannot access 'output.txt': No such file or directory
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 18: 实例 7 执行结果

实例 8: 查看 pre-commit 钩子内容

用 cat 查看 .git/hooks/pre-commit 文件内容。

```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ cat > Makefile << 'EOF'
.PHONY: all clean
all:
    touch output.txt
clean:
    rm -f output.txt
EOF
make all
ls -la output.txt
make clean
ls -la output.txt
touch output.txt
-rw-rw-r-- 1 dorothea dorothea 0 Sep  7 19:52 output.txt
rm -f output.txt
ls: cannot access 'output.txt': No such file or directory
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 19: 实例 8 执行结果

实例 9: 运行代码覆盖率工具

用 pytest-cov 运行测试并生成覆盖率报告。


```
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$ pip install pytest-cov --break-system-packages
pytest test_math.py --cov=. --cov-report=term
Defaulting to user installation because normal site-packages is not writeable
Collecting pytest-cov
  Downloading pytest_cov-7.1.0-py3-none-any.whl.metadata (32 kB)
Collecting coverage>=7.10.6 (from coverage[tonl]>=7.10.6->pytest-cov)
  Downloading coverage-7.16.0-cp314-cp314-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl.metadata (8.6 kB)
Requirement already satisfied: pluggy>=1.2 in /home/dorothea/.local/lib/python3.14/site-packages (from pytest-cov) (1.6.0)
Requirement already satisfied: pytest>=7 in /home/dorothea/.local/lib/python3.14/site-packages (from pytest-cov) (9.1.1)
Requirement already satisfied: iniconfig>=1.0.1 in /home/dorothea/.local/lib/python3.14/site-packages (from pytest>=7->pytest-cov) (2.3.0)
Requirement already satisfied: packaging>=22 in /usr/lib/python3/dist-packages (from pytest>=7->pytest-cov) (26.0)
Requirement already satisfied: pygments>=2.7.2 in /usr/lib/python3/dist-packages (from pytest>=7->pytest-cov) (2.19.2)
Downloading pytest_cov-7.1.0-py3-none-any.whl (22 kB)
Downloading coverage-7.16.0-cp314-cp314-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl (257 kB)
Installing collected packages: coverage, pytest-cov
Successfully installed coverage-7.16.0 pytest-cov-7.1.0

===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/dorothea/hw1/hw4_practice
plugins: cov-7.1.0, typeguard-4.4.4
collected 1 item

test_math.py . [100%]

===== tests coverage =====
_____ coverage: platform linux, python 3.14.4-final-0 _____

Name           Stmts  Miss  Cover
-----
test.py          4      4     0%
test_code.py     1      1     0%
test_math.py     5      0    100%
-----
TOTAL             10      5     50%

===== 1 passed in 0.03s =====
dorothea@dorothea-VirtualBox:~/hw1/hw4_practice$
```

图 20: 实例 9 执行结果

实例 10: CI 检查脚本

编写 ci_check.sh, 模拟 CI 运行 ruff check 和 pytest。

```

./ci_check.sh
Running CI checks...
I001 [*] Import block is un-sorted or un-formatted
--> test.py:1:1
|
1 | import os, sys
| ^^^^^^^^^^^^^^^
help: Organize imports
|
- import os, sys
1 + import os
2 + import sys
3 |
|

F401 [*] `os` imported but unused
--> test.py:1:8
|
1 | import os, sys
|      ^^
help: Remove unused import
|
- import os, sys
1 |
|

F401 [*] `sys` imported but unused
--> test.py:1:12
|
1 | import os, sys
|      ^^^
help: Remove unused import
|
- import os, sys
1 |
|

F821 Undefined name `subprocess`
--> test_code.py:1:1
|
1 | subprocess.Popen("ls", shell=True)
| ^^^^^^^^^^^^^
Found 4 errors.
[!] 3 fixable with the '--fix' option.

```

图 21: 实例 10 执行结果

三、解题感悟

本次实验涵盖了代码质量工具、构建系统、API 数据处理和综合修复四个主题。

第 13 题让我把 ruff 和 pytest 集成到项目中，通过 check.sh 把格式化、检查、测试串成一条线，形成了本地质量门禁。第 14 题通过 Makefile 练习了增量构建，理解了依赖变化时只重建受影响产物的机制。第 15 题用 curl 和 jq 处理 API 数据，生成了 Markdown 报告，体验了命令行工具组合使用的便利。第 16 题模拟了一次完整的修复流程：从故意破坏代码、确认测试失败，到修复、构建 wheel 并计算 SHA-256。

课后 10 个实例练习了 ruff、pytest、pre-commit、sed、json 处理、Makefile、覆盖率测试等工具，这些在开发流程中都比较常用。整体来说，这一周的内容让我对代码质量保障和自动化构建有了更具体的认识。